1. IDENTIFICACIÓN DE LA GUÍA

Nombre de la guía:	Despliegue de modelo clasificador de señales
Código de la guía (No.):	002
Taller(es) o Laboratorio(s) aplicable(s):	Laboratorio O
Tiempo de trabajo práctico estimado:	3h
Asignatura(s) aplicable(s):	integración ML Embebido y Edge Computing
Programa(s) Académico(s) / Facultad(es):	Maestría en Automatización y Control Industrial

COMPETENCIAS	CONTENIDO TEMÁTICO	INDICADOR DE LOGRO
Despliegue de modelo de aprendizaje automático clasificador de señales en microcontrolador.	Adquisición y etiquetado de señales. Entrenamiento y prueba de modelos. Despliegue de modelo.	Evaluación de desempeño de modelo entrenado sobre microcontrolador.

2. FUNDAMENTO TEÓRICO

Un acelerómetro IMU (unidad de medición inercial) mide la aceleración lineal, la inclinación y la vibración a lo largo de tres ejes para rastrear el movimiento y la orientación. Como componente crucial de las IMU (a menudo combinadas con giroscopios), detectan las fuerzas de aceleración y la gravedad, y sirven como entrada principal para la navegación, la estabilidad y el reconocimiento de gestos en teléfonos inteligentes, automóviles y robótica. Como generalidades sobre los acelerómetros, tenemos:

- *Aspectos clave de los acelerómetros IMU Funcionalidad*: Miden la tasa de cambio en la velocidad (aceleración lineal) y pueden determinar la inclinación detectando la dirección de la gravedad terrestre.
- *Tecnología (MEMS)*: La mayoría de los acelerómetros IMU modernos son sistemas microelectromecánicos (MEMS), que utilizan una pequeña "masa de prueba" sobre resortes para detectar el movimiento a través de cambios en la capacitancia (ver figura 1).

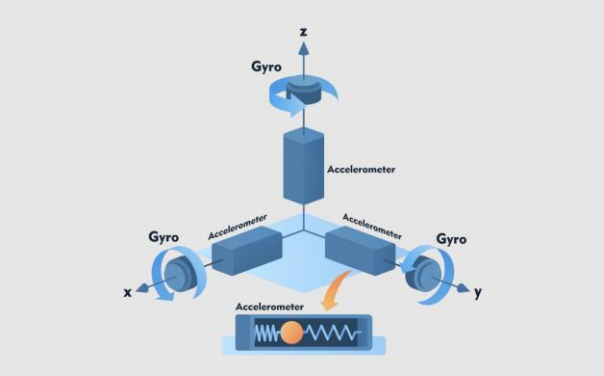

Figura 1. Esquemático de una unidad de medida inercial (IMU).

- *Grados de libertad (DoF)*: Un acelerómetro de 3 ejes proporciona 3 DoF. Cuando se combina con un giroscopio de 3 ejes, forma una IMU de 6 DoF. Al añadir un magnetómetro de 3 ejes se crea una IMU de 9 DoF para un seguimiento completo de la orientación.

Aplicaciones:

- *Electrónica de consumo*: Teléfonos móviles para la rotación de la pantalla, el recuento de pasos y los juegos.
- Industrial/automoción: monitorización de vibraciones para el mantenimiento predictivo, control de estabilidad de vehículos y drones.
- *Navegación*: sistemas de navegación inercial (INS) para determinar la posición y la velocidad.
- *Limitaciones y fusión de sensores*: Los acelerómetros son intrínsecamente ruidosos y sensibles a las vibraciones, lo que los hace inadecuados para el cálculo de la posición a largo plazo por sí solos. Normalmente se combinan con giroscopios (que miden la velocidad angular) utilizando algoritmos (por ejemplo, el filtro de Kalman) para cancelar la deriva y el ruido, proporcionando datos precisos y estables.

2.1. Adquisición de señales

Un aspecto importante al momento de adquirir señales es seleccionar adecuadamente la frecuencia de muestreo. Según el teorema de Nyquist la frecuencia de muestreo debe ser **mínimo el doble de máxima frecuencia de la señal** que se quiere adquirir. Por lo general, los movimientos que se van a adquirir son de frecuencias bajas, se puede emplear entonces frecuencias de muestreo mayores a 60Hz.
Otro dato relevante en la adquisición de señales es la resolución en bits, rango de valores y valor de representación mínimo. Identificar en la documentación del dispositivo cuál es la resolución del sensor del acelerómetro.

3. OBJETIVO(S)

3.1. Objetivo general

Entrenar y desplegar en microcontrolador un modelo basado en redes neuronales para la clasificación de gestos provenientes de señales de acelerómetros.

3.2. Objetivos específicos

3.2.1. Construir una base de datos con igual cantidad de duración para cada clase.
3.2.2. Extraer características de ventanas de las señales basado en análisis espectral.
3.2.3. Entrenar un modelo de clasificación utilizando redes neuronales.
3.2.4. Desplegar y validar el funcionamiento del clasificador en un microcontrolador.

4. RECURSOS REQUERIDOS

Plataforma Edge Impulse https://www.edgeimpulse.com/.
Computador con Arduino IDE https://www.arduino.cc/en/software/.
Microcontrolador Arduino Nano 33 BLE Sense.
Cable USB a USBC para programar el microcontrolador.

5. ASPECTOS DE SEGURIDAD

Evitar estar cargado electrostáticamente al momento de manipular los microcontroladores.

6. PROCEDIMIENTO O METODOLOGÍA PARA EL DESARROLLO

En esta guía describe los pasos para realizar la clasificación de señales de acelerómetros utilizando la plataforma Edge Impulse. El primer paso es la configuración del microcontrolador para adquirir las señales y enviarlas a la plataforma.

6.1. Construcción base de datos

6.1.1. Programación microcontrolador para adquisición

Pueden emplear el siguiente código y modificar la frecuencia de muestreo a su preferencia. Este código imprime en el puerto serial los valores del acelerómetro en los ejes "x","y" y "z".
https://docs.edgeimpulse.com/tools/clis/edge-impulse-cli/data-forwarder.

```
#include <Arduino_LSM9DS1.h>

#define CONVERT_G_TO_MS2   9.80665f
#define FREQUENCY_HZ       50
#define INTERVAL_MS        (1000 / (FREQUENCY_HZ + 1))

static unsigned long last_interval_ms = 0;

void setup() {
  Serial.begin(115200);
  Serial.println("Started");

  if (!IMU.begin()) {
    Serial.println("Failed to initialize IMU!");
    while (1);
  }
}

void loop() {
  float x, y, z;

  if (millis() > last_interval_ms + INTERVAL_MS) {
    last_interval_ms = millis();

    IMU.readAcceleration(x, y, z);

    Serial.print(x * CONVERT_G_TO_MS2);
    Serial.print('\t');
    Serial.print(y * CONVERT_G_TO_MS2);
    Serial.print('\t');
    Serial.println(z * CONVERT_G_TO_MS2);
  }
}
```

6.1.2. Almacenar base de datos en plataforma Edge Impulse

Existen varias formas para adquirir las señales y almacenarlas en la plataforma Edge Impulse:

- *WebUSB*: permite enviar datos desde el navegador https://edgeimpulse.com/blog/collect-sensor-data-straight-from-your-web-browser/.
- *Edge Impulse CLI – Data Forwarder*: como parte de la interfaz de línea de comandos (CLI) de Edge Impulse, se incluye un comando para conectar el dispositivo a la plataforma. Esta forma requiere instalar NodeJS, https://docs.edgeimpulse.com/tools/clis/edge-impulse-cli/installation. La información oficial de Data Forwarder está disponible en https://docs.edgeimpulse.com/tools/clis/edge-impulse-cli/data-forwarder.

Crear un proyecto público en Edge Impulse. Realizar el enlace entre el microcontrolador y Edge Impulse utilizando alguna de las formas descritas antes, o siguiendo la guía https://github.com/cgl-itm/AAEmbebido/blob/main/Tutoriales/DataForwarder_EdgeImpulse_example.md.
Seleccionar del menú de la izquierda en Edge Impulse la opción de "Data acquisition" (ver figura 2), desde allí empezaremos a tomar muestras y construir la base de datos.

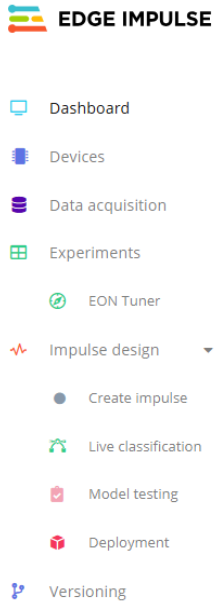

Figura 2. Menú de Edge Impulse.

En la sección "Collect data" que se muestra en la figura 3. Editar el campo "Label" y escribir el tipo de movimiento que se va a registrar, configurar el tamaño del registro en "Sample length (ms.)". Presionar el botón "Start sampling" para tomar el registro del movimiento.

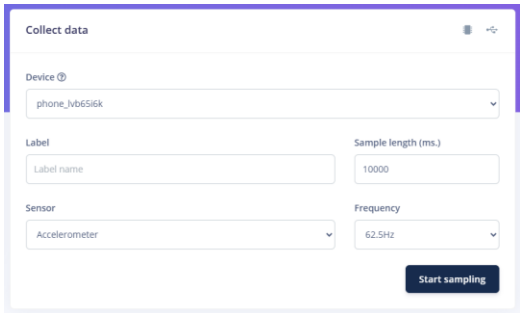

Figura 3. Sección "Collect data" de "Data acquisition".

Procedemos a almacenar tres tipos de movimientos diferentes. Los movimientos pueden ser: arriba-abajo, izquierda-derecha, frente-atrás. O movimientos que tengan formas: O, W, L, ∑, ∞.
Definir 3 tipos de movimiento o gestos diferentes. Tomar 20 registros de 10s por cada movimiento. Repetir el movimiento mientras se toma el registro. Adicionalmente, tomar 20 registros dejando el acelerómetro quieto (Idle). De esta forma se tendrán 4 clases (3 movimientos y quito).
Organizar el conjunto de datos para que sea 80% entrenamiento y 20% prueba.

6.2. Generación de tabla de características

Cuando trabajamos con señales en Edge Inpulse, diseñamos un Impulso el cual contiene 3 etapas para procesar los datos, como se muestra en la figura 4. Las dos primeras etapas son las que generan la tabla de características que recibe el clasificador de la tercera etapa.

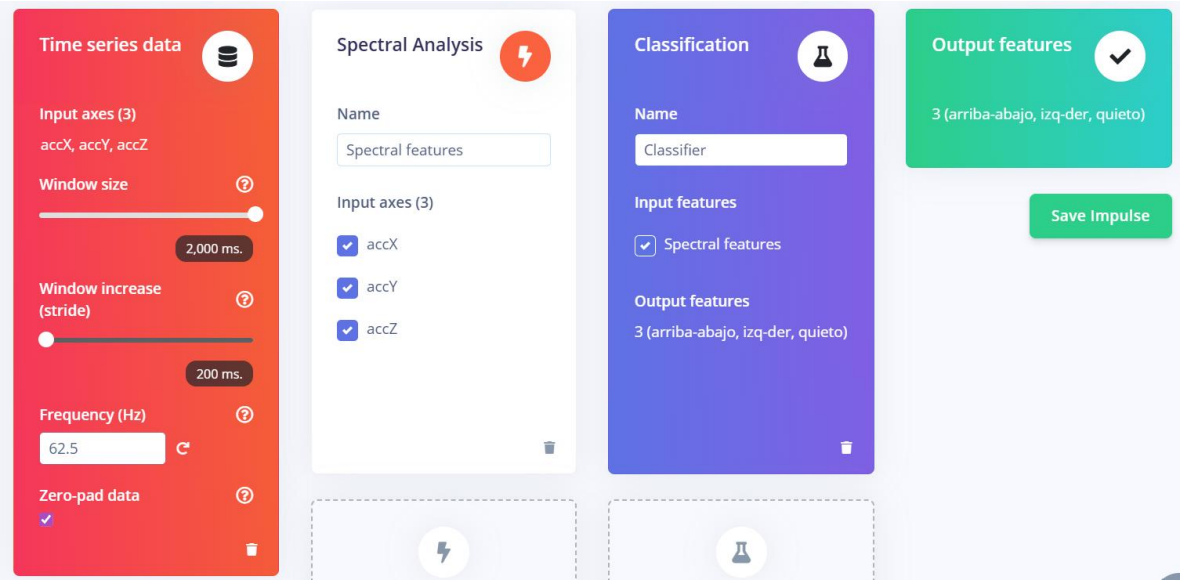

Figura 4. Diseño del impulso.

La etapa de "Time series data" consiste en como se van a procesar los registros tomados por los acelerómetros. Supongamos que tenemos registros de 10s, y configuramos el parámetro "Window size" en 2s y "Window increase" en 80ms. Entonces, vamos a extraer características espectrales sobre ventanas de tamaño 2s, con pasos de 80ms, como se muestra en la figura 5. La ultima ventana esta ubicada en el tiempo de 8s (porque 8s+2s=10s). **¿Cuantas ventanas se obtienen con esta configuración?**

Figura 5. Ventana deslizante con paso de 80ms.

En la sección de "Spectral features" del menú a la izquierda podemos observar el proceso de caracterización de las ventanas. Asumiendo que se tiene una frecuencia de muestreo de 62.5Hz y 3 canales de los acelerómetros, entonces para una ventana de 2s tenemos 125 datos por canal y 375 datos en total considerando todos los canales. Estos 375 datos son llamados "Raw features". Utilizando un análisis con la transformada rápida de Fourier (FFT) de tamaño 16 obtenemos 13 características por canal, y 39 en total por los 3 canales. Estas características se llaman "Processed features".

¿Cuánto se reduce el espacio inicial con respecto la cantidad de características procesadas?
¿De qué tamaño es la matriz de características que llega al clasificador para el entrenamiento?

6.3. Entrenamiento de modelo de clasificación

En esta etapa se empleará una red neuronal que la podemos configurar en la sección de "Classifier" dentro de "Impulse design" en el menú del lado izquierdo. La capa de entrada son las 39 características y la capa de salida corresponde a la cantidad de clases. Podemos agregar varias capas ocultas con diferentes tamaños.

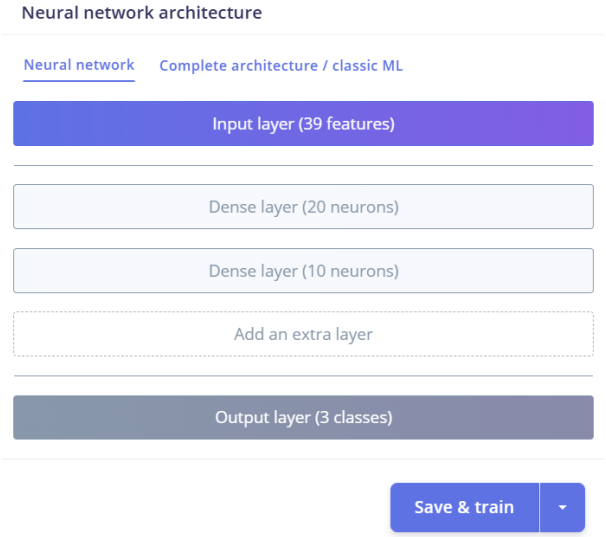

Figura 6. Configuración de la red neuronal.

En esta misma sección después de entrenar el modelo podemos analizar su desempeño empleando la matriz de confusión sobre el conjunto de validación.

¿Cuál método de extracción de características obtiene un mejor desempeño de clasificación FFT o Wavelet?

¿Con cuantas capas ocultas y cantidad de neuronas obtiene un mejor desempeño? (Justifique su respuesta).

6.4. Evaluación de modelo desplegado

En la sección de "Deployment" en el menú de la izquierda configuramos la forma en que necesitamos generar el código, desde el dispositivo hasta la maquina de inferencia, como se muentra en la figura 7.

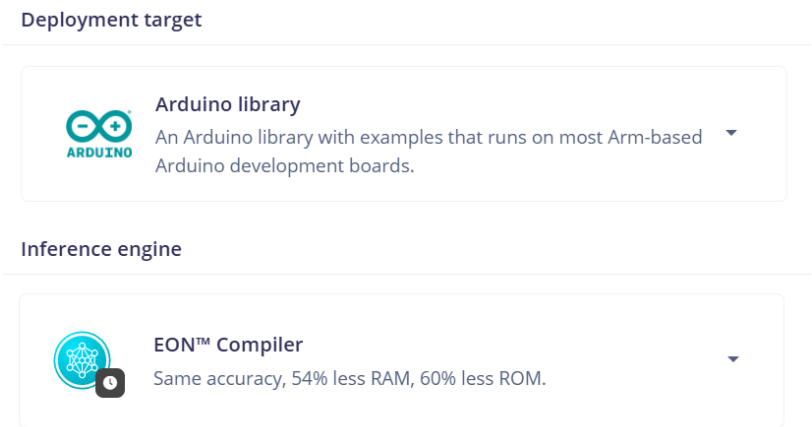

Figura 7. Configuración de la etapa de despliegue.

Después de seleccionar el objetivo del despliegue (en este caso librería de Arduino) presionamos en botón "Build" para construir el código del modelo. Cuando termine la construcción del modelo, descargamos el archivo ZIP, y la plataforma nos indica como agregamos el archivo como librería y podemos acceder a los códigos de ejemplo, como se muestra en la figura 8.

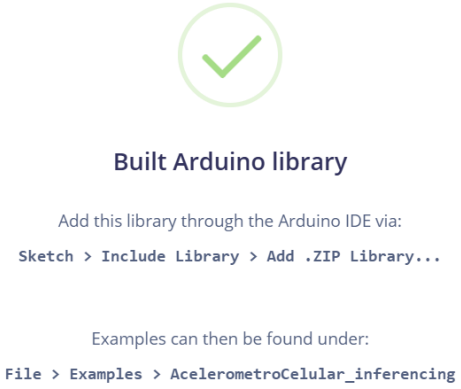

Figura 8. Descarga de modelo entrenado y explicación de configuración en Arduino.

7. PARÁMETROS PARA ELABORACIÓN DEL INFORME

Selección de la frecuencia de muestreo, calculo de la cantidad de muestras en la tabla de características, selección del método de extracción de características, selección del modelo de clasificación basado en redes neuronales.
Código de programación del modelo de aprendizaje automático implementado en Microcontrolador.

8. DISPOSICIÓN DE RESIDUOS

No aplica.

9. BIBLIOGRAFÍA

Documentación oficial del microcontrolador Arduino Nano 33 BLE sense. https://docs.arduino.cc/hardware/nano-33-ble-sense-rev2/.

Arduino nano 33 BLE Sense – Edge Impulse. https://docs.edgeimpulse.com/hardware/boards/arduino-nano-33-ble-sense.

Motion recognition with anomaly detection. https://docs.edgeimpulse.com/tutorials/end-to-end/motion-recognition.

Elaborado por:	*Cristian Guarnizo Lemus*
Revisado por:	
Versión:	*1.0*
Fecha:	*23-02-2026*